

<Chapter 1>

----- Mathematical Logic -----

What Does Mathematical Logic Solve (1.1)

(The Problem)

Natural language, as a tool for human communication, is inherently context-dependent. Understanding meaning is not limited to the literal definitions of words; it also involves tone, intention, and the cultural background shared by the speakers.

However, this very characteristic creates significant difficulties when language is used in mathematical and computational context.

Consider the following statement

"The server is down or the network is slow"

A human reader can usually infer the intended meaning from the context. But does this statement mean that exactly one of two conditions is true? Or does it allow both conditions to be true simultaneously?

This dual interpretation reveals a genuine semantic ambiguity in the structure of the sentence.

A machine, however, lacks the ability to infer context. I can not "read between the lines" or understand intentions from tone. For this reason, it became necessary to develop an alternative language that is precise and rigorous, eliminating any possible ambiguity and defining every symbol in an unambiguous manner.

Definition and Essence (1.2)

- Mathematical logic is a branch of mathematics concerned with the study of formal reasoning and proofs in a precise and systematic manner.

Its primary objective is to establish methods of reasoning whose validity can be determined independently of intuition, opinion or contextual interpretation.

The simplest and most fundamental of these systems is **propositional logic**, sometimes referred to as **sentential logic** or the **logic of truth values**.

It provides a formal framework for analyzing statements and determining how their truth values are affected by logical operations.

Propositional logic is founded upon a central principle known as the **principle of bivalence**, which states that every proposition possesses exactly one truth value, and only one. A proposition is either:

- True (T) — often represented by the value 1 in computer system.

- False (F) — often represented by the value 0 in computer system.

With in this framework, no intermediate truth values are permitted.

Concepts such a "uncertain" fall outside the scope of classical propositional logic.

This distinguishes it fundamentally from fuzzy logic, which allows truth to be expressed in varying degrees rather than as an absolute binary choice.

Basic Concepts: (1.3)

A **proposition** is any declarative statement that can be assigned a truth value. In other words, it must be possible to determine whether the statement is true or false.

For example:

- " $2+2=4$ " is a proposition because it is true.
- "The moon is a planet" is also a proposition because it is false.

In both cases, the statements possess well-defined truth values.

Not every sentence, however, qualifies as a proposition. Questions, commands, and exclamatory expressions are generally excluded from proposition logic because they cannot be classified as either true or false.

For instance:

- "What time is it?"

- "Close the door"

- "What a beautiful day!"

} Are not a propositional
logic

Logical Connectives

Individual proposition can be combined to form more complex propositions through the use of **logical connectives**. These connectives define relationships between propositions and determine the truth value of the resulting compound statement.

The most important logical connectives are:

Negation (Not / $\neg$) reverse the value

Conjunction (AND / $\wedge$) true only when both cases are true

Disjunction (OR / $\vee$) true when one of both cases are true

Implication (IF...THEN / $\rightarrow$) logical imperative link, which is the basis of the construction of the construction of proofs.

Truth Tables

A truth table is a systematic method used to determine the truth values of a compound proposition for every possible combination of truth values assigned to its constituent proposition.

Truth tables provide a complete and unambiguous description of the behavior of logical connectives. For this reason, they play a fundamental role in logic, mathematics, digital connectives, ~~the~~ circuit design, and computer science.

Logic Equivalence (1.4)

What Is Logical Equivalence?

Two compound propositions are said to be logically equivalent if they have identical truth values for every possible assignment of truth values to their variables.

In other words, regardless of the values assigned to the propositions involved, the two expressions always produce the same result.

Logical equivalence is denoted by the symbol:

$\equiv$

Thus, if two propositions P and Q are logically equivalent, we write:

$$P \equiv Q$$

This means that P and Q have exactly the same truth table.

Why Is Logical Equivalence Important?

In digital logic, you often simplify a circuit while preserving its behavior.

The same idea appears in mathematical logic.

If two expressions are logically equivalent, one may be replaced by the other without changing the meaning or truth values of the statement.

This principle is fundamental in:

- Proof construction
- Logical simplification
- Automated reasoning
- Digital circuit optimization

De Morgan's Laws

Among the most important logical equivalences are
De Morgan's Laws

$$\neg(p \wedge q) \equiv (\neg p \vee \neg q)$$

And

$$\neg(p \vee q) \equiv (\neg p \wedge \neg q)$$

These Laws state that negation can be distributed across a compound proposition by changing

- AND into OR
- OR into AND

while negating each individual proposition

Implication Equivalence

An implication (or conditional statement) can be logically expressed without the conditional operator ($\rightarrow$) by using a (combination of negation ($\neg$) and disjunction ($\vee$)). This fundamental relationship is known as the **Implication Law** (or Conditional-Disjunction Equivalence):

$$P \rightarrow Q \equiv \neg P \vee Q$$

This equivalence asserts that the statement "if p , then q " is logically identical to stating "Either p is false, or q is true". Both expressions yield a truth value of **False** if and only if p is true and q is false; otherwise, they are **True**.

Another essential equivalence involving implications is the **Contrapositive Law**, which states that a conditional statement is logically equivalent to its contrapositive:

$$P \rightarrow Q \equiv \neg Q \rightarrow \neg P$$

Rules of Inference (1.5)

Introduction

In propositional logic, it's often necessary to determine whether a conclusion follows logically from a set of premises. To accomplish this, logicians use formal pattern of reasoning known as **rules of inference**.

A rule of inference is a valid logical form that allows the derivation of a conclusion from one or more premises. If the premises are true and the rule is applied correctly, the resulting conclusion must also be true.

Rules of inference play a fundamental role in mathematical proofs, computer science, artificial intelligence, and formal verification systems. They provide a systematic method for constructing valid arguments and demonstrating the correctness of logical reasoning.

Unlike logical equivalence, which focuses on replacing an expression with another equivalent expression, rule of inference focus on deriving new statements from existing ones.

Importance of Rules of Inference

Rules of inference form the foundation of formal reasoning. They enable mathematicians and computer scientists to move from known facts to new conclusions in a logically valid manner.

By repeatedly applying these rules, complex arguments and mathematical proofs can be constructed from simpler statements.

Hypothetical Syllogism

Definition:

Hypothetical Syllogism is a rule of inference that allows two conditional statements to be combined into a single conditional statement.

If one proposition implies a second proposition, and the second proposition implies the third proposition, then the first proposition implies the third proposition.

Its logical form is:

$$((P \rightarrow Q) \wedge (Q \rightarrow R)) \Rightarrow (P \rightarrow R)$$

Importance

Hypothetical Syllogism is widely used in mathematical proofs and formal reasoning because it allows long chains of implications to be simplified into a single implication.

This rule is particularly useful when analyzing complex logical systems in mathematics, computer science, and software verification.

Modus Ponens

One of the most fundamental rules of inference is **Modus Ponens**.

Its logical form is:

$$(P \rightarrow Q) \wedge P \Rightarrow Q$$

In words:

1. If P implies Q .
2. P is true
3. Therefore, Q is true

Modus Tollens

Another important rule of inference is Modus Tollens

Its logical form is:

$$((P \rightarrow Q) \wedge \neg Q) \Rightarrow \neg P$$

In words:

1. If P implies Q
2. Q is false
3. Therefore, P is false

Disjunctive syllogism

Definitive

Disjunctive Syllogism is a valid rule of inference in Proposition logic. It states that if we are given a disjunctive (an "OR" statement) of two propositions. And we establish that one of the proposition is False, then we can logically conclude that the other proposition must
Its logical form is:

$$(P \vee Q) \wedge \neg P \Rightarrow Q$$

Alternatively:

$$(P \vee Q) \wedge \neg Q \Rightarrow P$$

Importance

Disjunctive syllogism is a cornerstone of problem-solving in computer science, specifically in algorithm design and debugging. When a system can fail for one of several reasons, systematically ruling out more causes automatically narrows down the logical possibilities, effectively guiding software engineers toward the root cause.

Resolution (Advanced)

Definition

Resolution is a powerful inference rule used primarily in automated theorem proving and artificial intelligence.

It operates on propositions that are expressed as disjunction (OR statements).

The rule states that if we have two disjunctions containing complementary literals (a proposition and its exact negation), we can resolve them by combining the remaining parts into a new valid disjunction.

Its logical form is:

$$((P \vee Q) \wedge (\neg P \vee R)) \Rightarrow (Q \vee R)$$

Application and Importance in Computer Science

Definition & Context

The principles of propositional logic are not merely abstract mathematical concepts; they are the foundational laws governing every computational system. Understanding these rules is what separates a standard programmer from a system architect or a cybersecurity engineer who can analyze hardware-level behavior and memory allocation safety.

1. Digital Logic Design (Hardware)

At the lowest level, processors execute instructions using logic gates. A CPU is essentially a massive network of interconnected propositions.

- **AND Gate**: Represents logical conjunction ($P \wedge Q$).
- **OR Gate**: Represents logical Disjunction ($P \vee Q$).
- **NOT Gate**: Represents logical Negation ($\neg P$).

2. Control Flow in programming

Every conditional statement in languages like C relies directly on propositional logic. When evaluating memory pointers or array bounds, engineers use logical connectives to prevent system crashes.

3. Cybersecurity and Software Verification

Formal verification uses logic to prove that software systems (especially operating systems and cryptographic protocols) behave correctly under all possible inputs. If a threat actor exploits a vulnerability, incident response teams use logical deduction (like Disjunction syllogism and Resolution) to trace the attack vector and establish the root cause.

Summary & Review

1. Logical Equivalences:

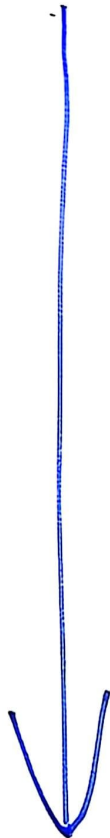
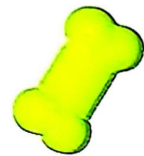
• Implication:

$$p \rightarrow q \equiv \neg p \vee q$$

• Contrapositive:

$$p \rightarrow q \equiv \neg q \rightarrow \neg p$$

2. Rules of inference:



Rule Name	Premises	conclusion
Modus Ponens	$P \rightarrow Q$ P	Q
Modus Tollens	$P \rightarrow Q$ $\neg Q$	$\neg P$
Hypothetical Syllogism	$P \rightarrow Q$ $Q \rightarrow R$	$P \rightarrow R$
disjunctive Syllogism	$P \vee Q$ $\neg P$	Q
Conjunction	P Q	$P \wedge Q$
simplification	$P \wedge Q$	P
Addition	P	$P \vee Q$
Resolution	$P \vee Q$ $\neg P \vee R$	$Q \vee R$